

Software Development for Disney Animated Feature Film Production

Rajesh Sharma, Brian Wherry
Walt Disney Animation Studios
Burbank, USA

rajesh.sharma@disney.com, brian.wherry@disney.com

Abstract

This report describes our experience in adapting the Scrum process to develop complex tools and user interfaces for use in computer-generated (CG) animated feature film production at the Walt Disney Animation Studios. We describe the movie making process and how the need for tools with good interaction led us to adopt a Scrum-based iterative design and development process.

1. Introduction

1.1 A primer on CG animated film production

Feature-length animated film production is a complex endeavor that takes a crew of hundreds many years to complete. The story is iterated upon by the director and story artists using “story reels” which are actual movies made from storyboard images, scratch dialog, sound effects, and temp music. The story reels give the filmmakers and screening audiences an approximation of “the movie,” albeit with quick sketches for visuals instead of finished frames. Once the story is in good shape (or once the release date is about a year out, whichever comes first) the CG production begins. In addition to using many third party tools, we develop custom software in-house for each stage of the production process.

The first step in the CG production process is modeling. Every character, prop, and environment needs to be modeled in 3-D space using a 3-D modeling program. Some models are quite basic, like a simple sphere for a tennis ball, whereas a spiny dinosaur model would be very complex, with a variety of geometric shapes combining to form the overall surface.

In addition, a rig is created for every character model to enable it to be animated. A rig is a skeleton of bones and movement controls that is inserted into the model. Animators use the rig to control the characters' movements. Although we use a third party tool for modeling, we build custom plug-ins and helper applications for making the modeling and rigging artists' lives easier. We also build and maintain systems for revision control of these 3-D assets.

The next step is layout, which is the process of arranging the characters and props into sets, along with cameras to capture the action – just as would be done in a live action movie, but all inside the computer. The layout artists also put in “blocking” animation, which is when the characters are simply moved to roughly the positions they will be occupying once they're animated later on. As with modeling, the bulk of the layout work is done within the 3-D modeling program, but we develop additional tools in-house for various tasks. One such task is splitting a long, continuous sequence, complete with blocking animation and camera moves and switches, into multiple shots (and multiple 3-D scene files) to be animated individually.

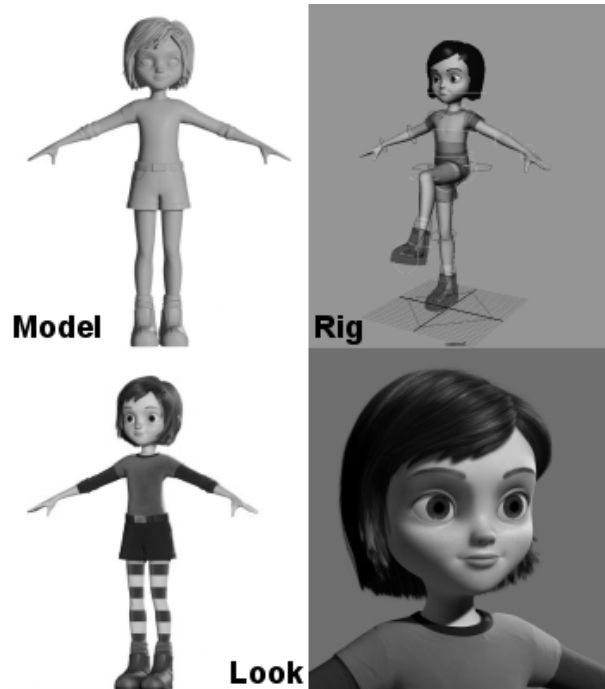


Figure 1. Progression of Penny element from *Bolt*

Another process runs in parallel with layout: look development. At this stage there's 3-D geometry, but to make a sphere look like a tennis ball it needs “look data.” The look of a CG element is defined by its surface properties, such as its color, its texture, the way it reflects light, and any patterns or graphics it may have (such as graffiti on a brick wall). Look development artists create look data for all the

elements once they've been modeled. Everything from rusty metal to shiny plastic to a graffiti-covered wall must all be designed, programmed, and painted. We develop and maintain a large suite of software tools for these look development tasks, some of which we'll talk about later.

Next comes animation. Animators position and articulate every part of every character, frame by frame, including moving each character's lips to match the dialog, blinking the eyes, and so on. We develop software to interface with the character rigs, enabling the animators to essentially directly manipulate a marionette, which in turn results in the CG model of the character animating as requested.

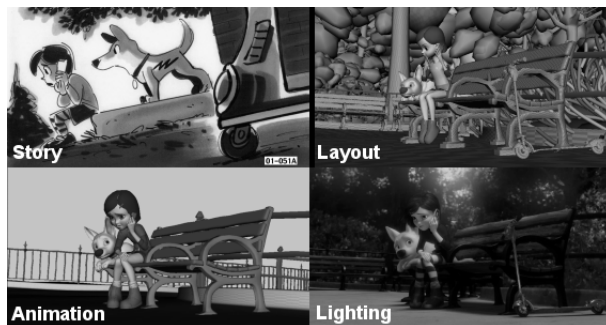


Figure 2. A typical shot progression

The last stage of the process is lighting. Up until this point all of the CG elements are laid out in scenes, looks have been developed for all the surfaces, and the characters are animated. For the audience to be able to see all that hard work up on the big screen these scenes need to be lit. Lighting artists add lights to the scenes to set the mood and highlight the action, not unlike the real lights in a theater production or live action movie. The lighting artists' job is complex because they're working with all the data in the scene at once. Whereas an animator only works with one or two characters per shot and a look development artist is only concerned with a single element at a time, the lighting artist must incorporate all the look data for every element that's been laid out and/or animated in the scene in order to apply appropriate lighting. Once lights are added to the scene, the lighting artist performs test renders. Rendering is the process of creating a two-dimensional image of a 3-D scene from a certain camera's perspective, applying all the necessary look data and lighting.

The final images used in the movie are rendered images. We develop and maintain a suite of tools for performing look development, lighting and rendering. These tools comprise what we call the render pipeline. Render pipeline development is the basis of this experience report.

1.2 The need for a CG tool makeover

At Walt Disney Animation Studios (WDAS), various departments, such as story, modeling, layout, rigging, animation, look, effects and lighting, work on data using tools specific to their department and pass the modified data as input to the next department in the pipeline.

Our current pipeline and tools have evolved over years of movie making with some ideas going back decades to the hand-drawn animation era. The current pipeline had reached the dreaded "5.0" release, where technical debt had caused many unfortunate couplings, data dependencies, reliance on naming conventions, component duplication and overlap, etc. Some parts of the old system became so complicated that some users didn't even use them – they did other things that "got close" instead.

In addition, much inefficiency had crept into both the process and tools over the years, leading to long render times. Long render times slow down production, especially in lighting.

It was apparent that we needed to rethink the workflow for each department and redesign the toolset around it. We needed a toolset that enabled faster turnaround on renders, quicker iterations on artists' work, was more intuitive to use, and required less training for new artists. The goal would be to let the artists do more in less time, preserving release dates and saving money in the process.

1.3 An opportunity for change

For the first time in nearly a decade, Walt Disney Animation Studios is producing a hand-drawn animated feature, *The Princess and The Frog*, which will arrive in theatres in November 2009. *Rapunzel*, the next slated CG feature, was moved to a holiday 2010 release. As *Frog* is a hand-drawn, 2-D production, it will not use any part of the render pipeline. Additionally, the software department has undergone some organizational changes in the past couple of years, allowing for more forward-thinking development projects, with less emphasis on simply reacting to the immediate needs of the current production. The combination of these factors gave our team the perfect opportunity to address some of the aforementioned problems with our render pipeline software.

With a project of this magnitude it was crucial that we kept the old system running at all times so that existing work could continue without interruptions while allowing artists to test and try out the new and improved functionality.

2. Trust the process

A team of seven developers with different areas of expertise was formed to undertake a pilot project to test the feasibility of replacing the existing render pipeline. The primary goals of the pilot project were to get a good handle on technical requirements and the scale of the required effort, and to gauge the support and commitment of stakeholders for such a large change.

Previously we did not have a formal development process in place. A couple of us had read the original Scrum paper from OOPSLA '95 and thought that it would be good process to follow given project management requirements. The description of Scrum seemed to match well against our needs. As all of our users are in-house, we would be able to quickly and easily leverage their feedback. Developing in short time boxes, Scrum's sprints, would let us get changes and improvements into the system fast.

No one in the organization had any prior experience of actually using the Scrum process in its entirety on a real project. We tried out some Scrum practices, such as the daily meeting, but our backlog was just a long to-do list. We hadn't broken down the backlog items to enough granularity, and individual developers owned certain areas of development as opposed to team ownership. We quickly realized that what served us well for the short-lived (but successful) pilot project would not be sufficient once we started work on the full project.

We decided that it would be wise to get formally trained as a group in the 'way of the Scrum.' We were lucky to be taught a two-day crash course by one of the originators, Jeff Sutherland. Our manager also attended this course. After the training, we picked out people to fill the roles of Product Owner and Scrum Master, found a common area for daily meetings with a permanent location for the publicly visible Scrum board, and collected planning poker cards, post-its and markers. We also created a wiki with more details on the individual scrum tasks.

One of the important touchstones of the project was to maintain transparency and accountability at all times. With a large in-house user base, production leadership and management needed to have a clear vision of where we were heading and how were going to get there. With millions of dollars to be made or lost on the theatrical release alone, we certainly didn't want to be responsible for production grinding to a halt due to this project. We also wanted to make sure we didn't try to tackle too much, and the technical leadership on *Rapunzel* was kept in the loop on all development activities. We needed to show results and steady

progress via regular demos and deliverables to ensure our stakeholders' awareness of our progress. Finally, it was critical that we didn't remove any existing tools without first providing equivalent (and hopefully better) functionality.

3. Importance of good interaction design

The most complicated part of the production process is the final lighting and rendering. A seemingly simple scene with two characters running through a forest is really thousands of tree models, each containing hundreds of branches and thousands of individually modeled leaves. Every surface in the scene has its own look data, and all of it needs to be lit and rendered.

With a "heavyweight" scene such as this forest scene, the lighting artist's interaction with it within the 3-D geometry viewing application is a slow, painful one. There are so many 3-D elements in it that just changing the frame of view can take many seconds for the application to respond. The artists do not have immediate, interactive feedback on their lighting decisions. They must make an informed guess as to where to place lights and how to set their colors, intensities, and other properties. Then they perform a low-resolution test render, which takes anywhere from thirty seconds to two hours to complete. After they see the test render they can move lights in space, adjust intensities and colors, tweak properties, and do another test render. They repeat this loop until it looks good enough to show the art director and/or director, then it's time to move on to the next shot (or potentially time to go back to the drawing board).

Given this inescapably laborious lighting process, if the tools available to the artists are difficult to use, don't make sense, and/or don't match the artists' conceptual model of what they're doing to the scene, we've got big problems. The process is so complex and time consuming to begin with that anything we can do to make the toolset more intuitive will lead to quicker shot turnaround and even more importantly, happier artists. In addition, the studio often needs to hire many additional lighting artists as the production gets closer to the release, to compensate for delays in upstream departments. A more intuitive toolset for surfaces, lighting and rendering will enable these new artists to get up and running more quickly, with minimal training, no matter what studio they've worked at previously.

Saving a mouse click here and there adds up to a lot of saved time over the course of a production. Parameter editing controls living where the artists expect to find them saves even more time. Tools doing

something closer to what the artists expect will result in fewer iterative, test renders, saving yet more time and artist frustration. Our ultimate goal is to enable the artists to remain in their artistic mindset while they're working, without having to be distracted by all the technical stuff. And with any luck we can help save a few dollars from the budget, too.

4. Balancing interaction design and software development

4.1 Research and design

We started our interaction design process by walking through high-level steps/tasks with artists to help us understand their artistic and technical needs. We then inspected our existing tools with the artists to try to determine how well they did or didn't address those tasks. This process helped us discover the artists' pain points. As we were looking to design tools and systems to span many departments, we also worked on identifying common interactions and similar tasks across departments, to guide us toward a common look and feel.

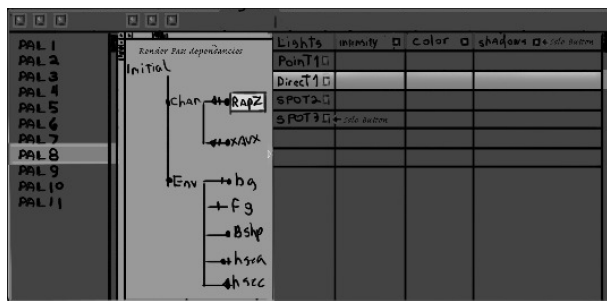


Figure 3. An early design sketch

Once we had a good handle on this initial information-gathering phase, we started designing the new system. We created workflow scripts and descriptions, sketched out whiteboard designs, and made paper prototypes with which to conduct usability tests with the artists. Despite our best efforts to follow interaction design guidelines, these methods were not as effective as we had hoped they would be. We discovered that, for the most part, our artists are very interested in (and critical of) the aesthetics of the visual design. Our sketches and paper prototypes were met with concern from some artists and were dismissed entirely by others. We needed to find a way to effectively design the artists' interactions with the system while incorporating their feedback early on.

4.2 Expert users as designers

During our R&D phase, a group of production artists and technical supervisors began a weekly

meeting at which they discussed ways to improve the lighting and rendering workflow and toolset. They aimed to identify user pain points in hopes of solving them for *Rapunzel*. Called the render pipeline task force, this group of people included former software developers, "power users" of our existing render pipeline tools, and some folks with product design experience.

One of the task force members, a lighting supervisor named Josh, is a Photoshop master with interest in GUI design. He offered to create nice-looking mockups of our ideas, incorporating the ideas of the task force. He actually went a step further and built a sort of "concept car" using link-mapped images in HTML pages, resulting in an approximation of how the entire application will look and behave. Once the task force had Josh's blueprint, their discussions became much more focused, as they had a tangible representation of what they ultimately wanted. Josh was able to capture their input and constantly evolve the blueprint.



Figure 4. A part of the working blueprint

At one point the render pipeline task force was meeting multiple times a week, hashing out workflow and design ideas as the scope expanded from look development tools to lighting, to render settings, and so on. Josh was updating the blueprint every step of the way to capture the discussion into a living, tangible form. During this time the render pipeline development team was Scrumming along, focused on the current sprint tasks. To make sure the task force wasn't totally going haywire, a member of the development team attended the task force meetings. The render pipeline task force's discoveries in their discussions and critiques of the blueprint would not only force them to rethink their workflow for different tasks within the application, but also help them prioritize their needs, which informed the product owner's prioritization of the product backlog. Once the development team finished the current sprint and came up for air, we would all get up-to-date on the latest revisions to the blueprint and figure out what we would be working on during the next sprint.

This approach has been working very well for us, as the users figure out what they really want (and what

they want it to look like) while the development team can stay focused on sprint tasks.

4.3 Do it over ‘til you get it right

One of the new technologies that will enable faster renders for *Rapunzel* and beyond is the new shader system available in Renderman 14, the latest version of the rendering program we use. A shader is a part of the look for a 3-D element. Shaders are little programs, or modules, that determine what the 3-D geometry will look like when rendered. Shaders handle all properties such as the color of a surface, its texture, and how it reflects light.

Our old shaders were fine in their supply of artistic controls, but suffered from over complexity for simple tasks. That is, you got all the parameters, all the time. As more and more parameters were added to a shader for more complicated surface properties, the shader would take longer to do its job, and renders would take longer to complete. And since there was a single, monolithic shader available, a simple gray plastic ball would take the same time to render as a pearlescent giant glowing marble sphere of doom. With the new shader system, called modular co-shaders, you start with a blank slate and add what you need. So a plastic ball would be very simple, and would render very quickly. A complex orb of wonders would require many more shader components, and thus would take longer to render.

What modular co-shaders lack in overhead they make up for in complexity. The old shaders were overkill for simple tasks, but the interface for modifying shader parameters was a simple categorized, flat list (albeit a very long one). All the shaders looked the same – they just had different values for certain things. Modular co-shaders are different. A modular co-shader starts as an empty skeleton, into which little modules are inserted, each of which has different parameters to be modified. Some modules can accept other modules as parameter inputs, or even chains of other modules. Allowing for the infinite possibilities of module building would require a user interface that’s far more complex than what was needed for the old shaders. One small, yet important piece of the render pipeline development was to create a user interface for editing modular co-shaders.

Our first iteration on the modular co-shader editing GUI focused on allowing modules to go wherever they were allowed to go. We used little module widgets that simply showed the module name. Double-clicking a module widget would open a dialog where its parameters could be edited. While this solution was functional, it wasn’t very usable, as the artists couldn’t look at a surface material as a whole and see what

parameters were modified where.



Figure 5. An early sketch of the modular co-shader editing UI

Our second iteration of the co-shader editing GUI added some parameter editing widgets to the module widgets. The idea was to put the most important parameters on the module widgets, with the remainder accessible via the parameter editing dialog, as before. While this solution was a step in the right direction, it was still difficult for artists to visually inspect a material to see what it was doing. Also, our decision to arrange the parameter editing widgets vertically within the module widget led to a lot of unused horizontal space and thus, a lot of vertical scrolling.

In our third iteration we made module “strips” where the module widget would have parameter editing widgets arranged horizontally. The user could choose exactly how many parameters he would like to see for each module type, and in what order he would like them to appear. With this solution the artists can quickly see what’s going on with their shader modules and easily edit the parameters they care about.

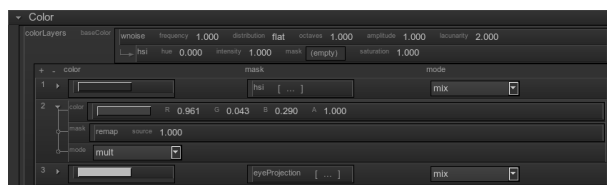


Figure 6. A portion of the final iteration of the modular co-shader editing UI

Each iteration of the UI was fully functional, but it took several passes to finally get it usable. Part of the reason it took three sprints for us to get it right was we were all still coming up to speed on how these modular co-shaders work, and what artists will want to do with them. Additionally, development of this particular piece of the render pipeline was done before the render pipeline task force had become so involved in the user interface and interaction design, and before the blueprint had been started.

5. Process improvements and lessons learned

5.1 Time boxing

Some of the design and development work required to finish a tool demanded too much effort and we ended up over-designing the interfaces to be implemented later. It occurred to us that interaction design needs to be agile too. We started time boxing design and prototyping. It allowed us to do something, demo it, get feedback quickly, adjust the designs and implement parts of it in an iterative cycle. This process let us explore pie-in-the-sky ideas and new possibilities instead of investing too much time on monolithic designs that we would be loathe to abandon. The ready availability of in-house users allowed us to test these iterative implementations even when the software was not production ready.

5.2 Swarming

As we were trying to avoid developers “swimming in lanes” by encouraging work on parts of the product outside their previous experience or comfort zone, we found great productivity increases when a small group of developers worked together on a related set of sprint tasks, consulting with a domain expert. We called it “swarming.” Swarming helped us finish tasks quicker and was very useful with cross training other team members in areas outside their expertise.

5.3 Dedicated support person

Support is critical when in-house users are involved. We pass these users everyday in the hallway. We started with the usual, Scrum-prescribed 20% of each team member’s time devoted to bug fixes and support. As more features got added to the tools and more users started using the toolset, it was decided to have one team member per week dedicated to support. The other team members reduced their 20% support so our velocity didn’t suffer. By having a different person on support every week, we also made sure that team members got to work on all aspects of the product. Dedicated support gave the production confidence in our ability to address their problems in a timely fashion. Also, meeting with them every other day established a working relationship and good communication.

5.4 Supporting other projects and varying schedules

Many of us work on other projects one day a week. Some of us work from home one day a week. By establishing a policy of notifying the team members of outages and availability early in the sprint, we avoided

the issue of de-fragmentation of availability. Production deadlines are set years in advance because of the movie release dates. For our software to be usable by the production we have to meet those deadlines well before the production departments start their work. The users have to be trained and need to feel comfortable with the new toolset before committing to use it. Our product backlog is available widely in the organization for anyone to add items to it. At the end of every sprint there are prioritization meetings with the product owner and key stakeholders, like production technical leadership, various department leads and users. The open backlog and team velocity allowed them to make hard decisions regarding features versus deadlines and prioritize accordingly.

6. Conclusion

At Walt Disney Animation Studios our mission is to create beautiful and timeless films through the art of storytelling, the magic of animation and the science of cutting edge technology. The software group develops and supports tools for a variety of users, from story artists, 3-D animators, modelers, lighters, painters, and shader developers to production staff who manage the flow of work through the pipeline. We are relative newcomers to Scrum but feel that the process suits the rapid pace of movie making, hard deadlines and frequent requirement changes.

Several challenges remain to be overcome. Something we would like to do in the future is to dedicate the last day of the sprint for larger functional tests that take a piece of data through the whole pipeline. For technical reasons, this cannot be automated yet, but we are exploring ideas to do this to add stability to our product.

As the software has grown in size, the number of unit tests has grown as well. Every integration task takes more time to complete, as tests take longer and longer to run. In our process we have reserved the week after the sprint for demos and planning for the next sprint, but often times we end up finishing and polishing items from the previous sprint. This creep sometimes prevents us from focusing on the next sprint’s tasks.

The stakeholders sometimes panic at the size of the backlog and switch the priorities too wildly, causing some thrashing in a way where there isn’t much continuity from one sprint to the next.

There is still room for improvement in the process and we hope that the process, just like our software, will iteratively evolve to meet our needs.